

Assignment: SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- Name: To Minh Ly
 - Student ID: 1123350
 - Course: AI-Assisted Software Development
 - Date: 2026/5/14
-

1. Introduction

AI-assisted software development means using AI tools to help developers plan, design, write, test, and improve software. AI tools can generate code, documentation, test cases, and explanations quickly. However, AI does not automatically know the real purpose of the project. If the requirement is unclear, the AI-generated result may be incomplete or incorrect.

Clear requirements are important because they help both developers and AI tools understand what should be built. In software development, unclear instructions may cause wrong features, missing validation, or incorrect calculations.

SDD, BDD, and TDD are useful in the AI era because they help developers guide and check AI-generated results. SDD helps define the system specification. BDD describes expected behavior through scenarios. TDD provides test cases to verify whether the result is correct.

2. Definition of SDD

SDD stands for Specification-Driven Development. It is a development approach that focuses on writing clear specifications before implementation.

A specification usually describes the goal of the system, functional requirements, input data, output data, constraints, and acceptance criteria. The purpose of SDD is to make sure developers understand what should be built before writing code.

In AI-assisted software development, SDD is useful because it can make AI prompts more specific. Instead of only asking AI to create a program, developers can provide clear rules and requirements.

3. SDD: Student Grade Calculator

3.1 Goal

The goal of the Student Grade Calculator is to calculate a student's final weighted score and assign a letter grade based on four components: assignment, midterm exam, final exam, and project.

The calculator should help users calculate grades consistently and reduce manual calculation mistakes.

3.2 Functional Requirements

The system should perform the following functions:

1. Accept four score components from the user.
2. Validate that every score is between 0 and 100.
3. Calculate the weighted final score.
4. Round the final score to one decimal place.
5. Convert the final score into a letter grade.
6. Display the final score and letter grade.
7. Show an error message if the input is invalid.

3.3 Input

The system requires four input values:

Input Item	Description	Valid Range
Assignment	Score for assignment work	0 to 100
Midterm Exam	Score for the midterm exam	0 to 100
Final Exam	Score for the final exam	0 to 100
Project	Score for the project	0 to 100

All input values should be numeric. Scores below 0 or above 100 should be treated as invalid.

3.4 Output

The expected output includes:

1. Final weighted score rounded to one decimal place.
2. Letter grade based on the final weighted score.
3. Error message if any input score is invalid.

Example output format:

- Final Score: 83.3
- Letter Grade: B

For invalid input:

- Error Message: Invalid input. Score must be between 0 and 100.
- Final Score: Not calculated
- Letter Grade: Not assigned

3.5 Grade Rules

The final score is calculated using this formula:

Final Score = Assignment × 0.30 + Midterm × 0.20 + Final Exam × 0.30 + Project × 0.20

The letter grade should follow these intervals:

Final Score	Letter Grade
90.0 to 100.0	A
80.0 to less than 90.0	B
70.0 to less than 80.0	C
60.0 to less than 70.0	D
Below 60.0	F

3.6 Acceptance Criteria

1. If all four input scores are valid, the system must calculate the final weighted score using the correct component weights.
2. The final score must be rounded to one decimal place before it is displayed.
3. If the final score is exactly 90.0, the system must assign letter grade A.
4. If the final score is exactly 60.0, the system must assign letter grade D.
5. If any score is below 0 or above 100, the system must show an error message and must not calculate the final score.
6. The system must display both the final score and the letter grade for valid input.

4. Definition of BDD

BDD stands for Behavior-Driven Development. It is a development approach that describes software requirements through user behavior and concrete scenarios.

BDD usually uses the Given-When-Then format:

- Given: the initial condition
- When: the action happens
- Then: the expected result should occur

BDD is useful because it makes requirements easier to understand. Developers, testers, users, and AI tools can all understand the expected behavior through simple scenarios.

5. BDD: Student Grade Calculator

Scenario 1: Student receives grade B

Given the assignment score is 82

And the midterm exam score is 78

And the final exam score is 85

And the project score is 88

When the system calculates the final grade
Then the final score should be 83.3
And the letter grade should be B

Scenario 2: Student receives grade D at the boundary

Given the assignment score is 60
And the midterm exam score is 60
And the final exam score is 60
And the project score is 60
When the system calculates the final grade
Then the final score should be 60.0
And the letter grade should be D

Scenario 3: Invalid score is entered

Given the assignment score is 70
And the midterm exam score is -5
And the final exam score is 82
And the project score is 77
When the system checks the input values
Then the system should show an invalid input message
And the final score should not be calculated

6. Definition of TDD

TDD stands for Test-Driven Development. It is a development approach where developers define test cases before or during implementation.

The purpose of TDD is to check whether the system works correctly. Developers first design tests, then implement the feature, and finally improve the code while keeping all tests passing.

The TDD cycle includes:

Step	Meaning
Red	Write a test that fails because the function is not implemented yet
Green	Write the simplest implementation that passes the test
Refactor	Improve the implementation while keeping tests passing

In this assignment, no program code is required. The focus is on designing test cases for the Student Grade Calculator.

7. TDD: Student Grade Calculator

Scenario 1: Normal Test Cases

Test Case 1: Calculate grade B

Input

- Assignment: 82
- Midterm: 78
- Final Exam: 85
- Project: 88

Expected Calculation

Final Score = $82 \times 0.30 + 78 \times 0.20 + 85 \times 0.30 + 88 \times 0.20$

Final Score = $24.6 + 15.6 + 25.5 + 17.6$

Final Score = 83.3

Expected Output

- Final Score: 83.3
- Letter Grade: B

Test Case 2: Calculate grade C

Input

- Assignment: 74
- Midterm: 69
- Final Exam: 76
- Project: 72

Expected Calculation

Final Score = $74 \times 0.30 + 69 \times 0.20 + 76 \times 0.30 + 72 \times 0.20$

Final Score = $22.2 + 13.8 + 22.8 + 14.4$

Final Score = 73.2

Expected Output

- Final Score: 73.2
- Letter Grade: C

Scenario 2: Boundary Test Cases

Test Case 1: Final score is exactly 90.0

Input

- Assignment: 90
- Midterm: 90

- Final Exam: 90
- Project: 90

Expected Calculation

Final Score = $90 \times 0.30 + 90 \times 0.20 + 90 \times 0.30 + 90 \times 0.20$

Final Score = $27.0 + 18.0 + 27.0 + 18.0$

Final Score = 90.0

Expected Output

- Final Score: 90.0
- Letter Grade: A

Test Case 2: Final score is exactly 60.0**Input**

- Assignment: 60
- Midterm: 60
- Final Exam: 60
- Project: 60

Expected Calculation

Final Score = $60 \times 0.30 + 60 \times 0.20 + 60 \times 0.30 + 60 \times 0.20$

Final Score = $18.0 + 12.0 + 18.0 + 12.0$

Final Score = 60.0

Expected Output

- Final Score: 60.0
- Letter Grade: D

Scenario 3: Invalid Input Test Cases**Test Case 1: Assignment score is above 100****Input**

- Assignment: 105
- Midterm: 80
- Final Exam: 75
- Project: 90

Expected Calculation

The assignment score is invalid because it is greater than 100.

The system should not calculate the final score.

Expected Output

- Error Message: Invalid input. Assignment score must be between 0 and 100.
- Final Score: Not calculated
- Letter Grade: Not assigned

Test Case 2: Midterm score is negative

Input

- Assignment: 70
- Midterm: -5
- Final Exam: 82
- Project: 77

Expected Calculation

The midterm score is invalid because it is below 0.

The system should not calculate the final score.

Expected Output

- Error Message: Invalid input. Midterm score must be between 0 and 100.
- Final Score: Not calculated
- Letter Grade: Not assigned

8. Comparison of SDD, BDD, and TDD

Item	SDD	BDD	TDD
Full Name	Specification-Driven Development	Behavior-Driven Development	Test-Driven Development
Main Focus	Requirements and system rules	User behavior and scenarios	Test cases and correctness
Main Question	What should be built?	How should the system behave?	How can we verify correctness?
Typical Format	Goal, input, output, constraints, acceptance criteria	Given-When-Then scenarios	Test case input, calculation, expected output
Main User	Developer and project planner	Developer, tester, and user	Developer and tester

Item	SDD	BDD	TDD
AI-era Value	Helps AI understand the exact requirements	Helps AI understand expected behavior	Helps check whether AI-generated output is correct
Example in This Project	Define score weights and grade rules	Describe grade calculation situations	Test normal, boundary, and invalid cases

SDD, BDD, and TDD are related but different. SDD explains what the system should do. BDD explains how the system should behave in different situations. TDD checks whether the system gives the correct result.

9. Reflection

Among SDD, BDD, and TDD, BDD is the easiest for me to understand because it uses real examples and simple language. The Given-When-Then format is clear. It shows the input situation, the action, and the expected result. This makes it easier to understand how the software should behave.

When working with AI coding tools, I think SDD is very useful at the beginning of a project. AI can generate code quickly, but if the prompt is unclear, the result may miss important rules. For example, if I only ask AI to create a grade calculator, it may not know the exact score weights, grade intervals, rounding rule, or invalid input rule. SDD helps reduce unclear prompts because it requires the developer to write the goal, input, output, constraints, and acceptance criteria first.

BDD is useful because it describes user expectations through concrete scenarios. Instead of only saying that the system should calculate grades, BDD shows examples of how the system should behave. This helps developers and AI tools understand the expected result more clearly.

TDD is important because AI-generated code still needs to be checked. Even if the code looks correct, it may fail in boundary cases or invalid input cases. TDD helps developers prepare test cases before implementation, so they can verify whether the output is correct.

In future software projects, I would combine these three approaches. First, I would use SDD to define the system requirements. Second, I would use BDD to describe important user scenarios. Third, I would use TDD to create test cases for normal cases, boundary cases, and invalid input cases. This can help me guide AI tools better and reduce mistakes in the final result.

10. References / AI Tool Usage

- ChatGPT was used to help organize the report structure and explain SDD, BDD, and TDD in simple words.
- ChatGPT was used to help create original BDD and TDD examples with score combinations different from the assignment examples.
- The final content was reviewed and adjusted by myself.
- Course assignment document: Assignment II - SDD, BDD, and TDD in AI-Assisted Software Development.